

TP N° 10

Algorithmes de tri

L'objectif est de comparer différents algorithmes de tri dans une classe permettant de manipuler des vecteurs de nombres entiers aléatoires réels.

- 1) Ecrivez une classe *CVect* comportant comme données membres privées (un entier *m_nN* représentant la taille d'un tableau et un pointeur *m_ptab* sur des entiers) et comme méthodes membres publiques (un constructeur avec paramètres par défaut permettant l'initialisation de *m_nN*, l'allocation dynamique du pointeur *m_ptab* et enfin l'initialisation du tableau *m_ptab* à 0).
N'oubliez pas le destructeur.
Dans votre fonction *main()*, instancier un objet de type *CVect* et tester votre constructeur.
- 2) Ajoutez à la classe *CVect* un 2^{ème} constructeur *CVect(int, double*)* ayant 2 paramètres, un entier et un pointeur sur des entiers pour initialiser *m_nN* et *m_ptab*.
Tester dans votre programme principal.
- 3) Redéfinir l'opérateur *ostream& operator <<(ostream &, const CVect &)* pour afficher un objet de type *CVect*.
Tester dans votre programme principal.
- 4) Ajouter une méthode membre *void Random()* permettant de générer des nombres aléatoires entre 10 et 20.
- 5) Compte tenu de la nature des données membres de la classe, surcharger toutes méthodes et tous opérateurs qui vous semblent nécessaires.

Tri par sélection

Un des premiers algorithmes de tri est le *tri par sélection*. Il consiste à trouver l'élément le plus petit dans le tableau et de le permuter avec le premier élément. Ensuite, on recommence avec le reste du tableau et ainsi de suite (voir exemple ci-dessous).

9	5	1	6	2
1	5	9	6	2
1	2	9	6	5
1	2	5	6	9
1	2	5	6	9

- 6) Ajouter à votre classe une méthode *void Permute(int i, int j)* permettant de permuter les éléments *i* et *j* du tableau *m_pTab*.
Tester dans votre programme principal.
- 7) A l'aide de la méthode *void Permute()*, compléter votre classe par une méthode *void TriSelection()* permettant de trier le tableau *m_pTab* par le tri sélection.
Tester dans votre programme principal.

Tri à bulles

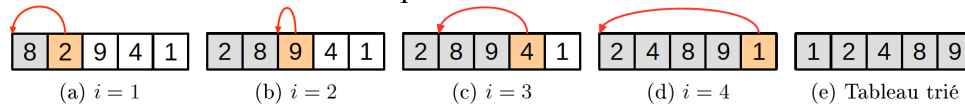
Le tri à bulles est sans doute l'algorithme le plus long en temps d'exécution. Il consiste à comparer successivement tous les éléments adjacents d'un tableau et de les échanger selon l'ordre souhaité (croissant ou décroissant). Cette opération est répétée tant que le tableau n'est pas complètement trié.

- 8) Ajouter à votre classe une méthode *void TriBulles()* permettant de trier le tableau *m_pTab* par l'algorithme à bulles.

Tester dans votre programme principal.

Tri par insertion

Le tri par insertion fonctionne comme le tri par sélection, la seule différence est qu'on s'intéresse à l'indice de l'élément à trier et non à la valeur de l'élément. Ainsi, dans le tri par insertion c'est l'élément non trié de plus petit indice et pas celui de valeur minimum qui est ajouté au reste du tableau déjà trié. Ceci peut nécessiter de décaler une partie des éléments triés pour insérer l'élément à trier à sa place.



Exemple : Tri par sélection d'un tableau. Les cases en gris clair représentent les éléments déjà triés. La case orange représente l'élément à placer à la bonne position parmi les éléments déjà triés.

- 9) Ajouter à votre classe une méthode *void TriInsertion()* permettant de trier le tableau *m_pTab* par insertion.

LE TP DOIT ETRE TERMINE POUR LA SEANCE SUIVANTE